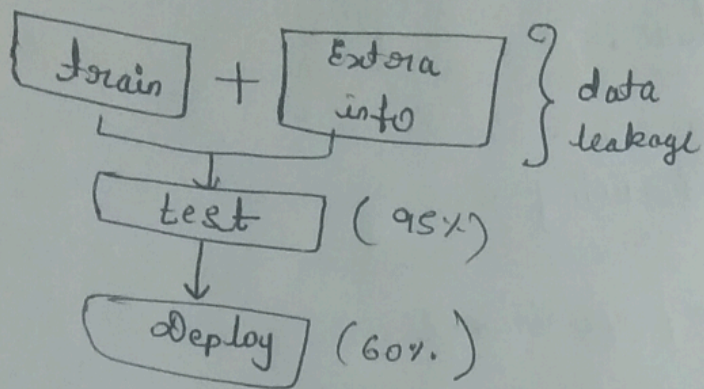


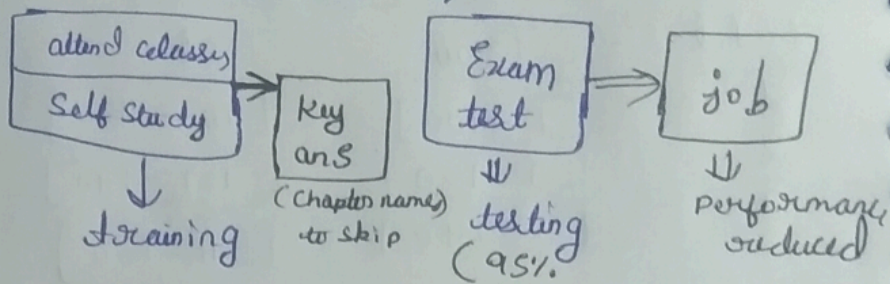
Data Leakage:-

Data leakage occurs when future or target-related information leaks into training, causing over-optimistic results.



Analogy:-

like
"Data leakage is giving students the ans key before Exam. They score high, but that score does not reflect on real knowledge."



Why is data leakage dangerous?

→ Data leakage is dangerous because it produces unrealistically good Evaluation metrics (results), causing models to fail in real-world deployment and leading to wrong decisions.

i) It gives fake accuracy:-

- model sees part of ans during training
- So it predicts well in testing
- but fails in real-world deployment

ii) model fails in production

- in real life, leaked info is not available
- model cannot reproduce test performance
- sudden performance drop after deployment.

iii) Wrong Business decisions

→ Because the model looks good

- fraud rules may block real users, vice versa

- medical diagnosis may miss patient.

financial losses increase

iv) Hard to detect

- No errors or warnings
- Code runs perfectly
- Only noticed after deployment

⇒ waste time & resources

- later discover the model is unusable & retraining from scratch

Ways in which data leakage can occur:-

Data leakage usually happens when information that should be unavailable at prediction time is used during training or evaluation.

i) Target leakage: [Most Dangerous]

A feature contains direct indirect information about the target that wouldn't be available at prediction time.

Ex:- Credit risk model

feature	loan-amt	total pymts-made	credit-utilization	loan-status
		data leaked		

→ loan-amt → fees only known after approval decision

→ total payments-made → Payment count includes post-prediction

→ credit-utilization → available at application time

ii) Multicollinearity with target col:-

Ex:- Smartphone price prediction

brand	RAM	weight	...	rating	price
-------	-----	--------	-----	--------	-------

high correlation with target

• usually costly phones are highly rated

iii) Train-Test leakage:- Test data influences training

Common mistakes

- Scaling before splitting
- Normalization on full dataset
- feature selection before split
- PCA before split

wrong
scale → split → train

correct

split → scale (fit on only train)
test (transform only)

Train-test leakage happens when preprocessing is applied before splitting the data.

iv) duplicate / Sort of duplicate

consider, 1000 dataset

↓
2000 rows are duplicated

↓
train model

↓
test

Since data's matching with training
accuracy improves

v) Data Preprocessing Leakage:

- imputing / filling missing values using full dataset mean
- Encoding categories using full dataset
- Standardizing full dataset

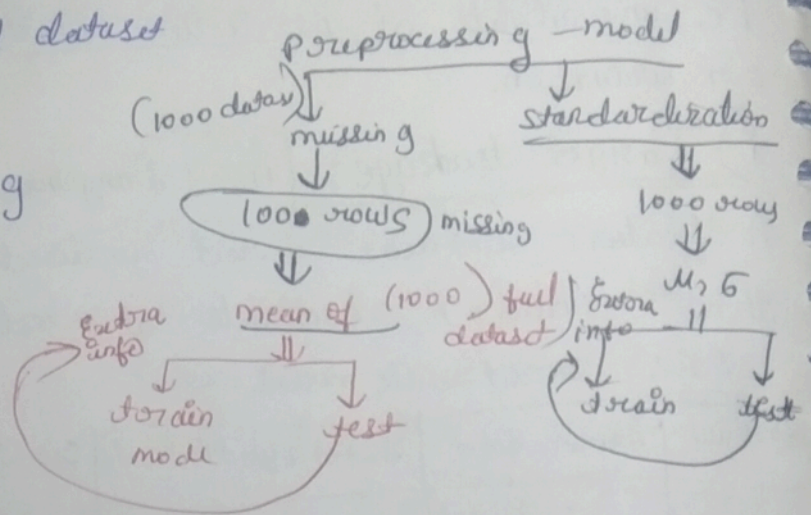
Test statistics influence training

vi) CV leakage:-

- Preprocessing is done outside the CV loop.

Ex:- Scaling full dataset

- Then applying K-fold CV



→ Each fold sees statistics from other folds

fix:- use pipelines

vi) Temporal Leakage [Time-based Leakage]

- future data is used to predict the past

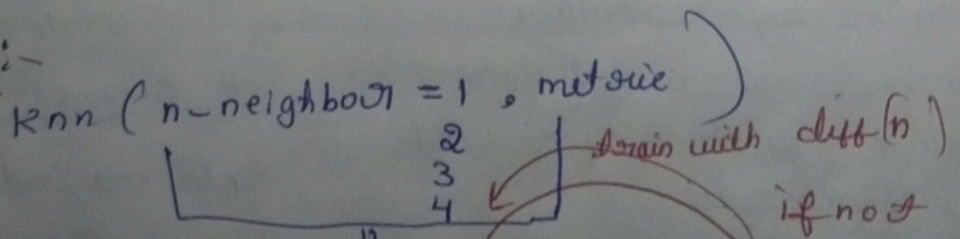
Ex:- Random split on time-series data

- Shuffling timestamped records

why its bad:- real-world prediction only have past data

vii) Hyperparameter tuning:-

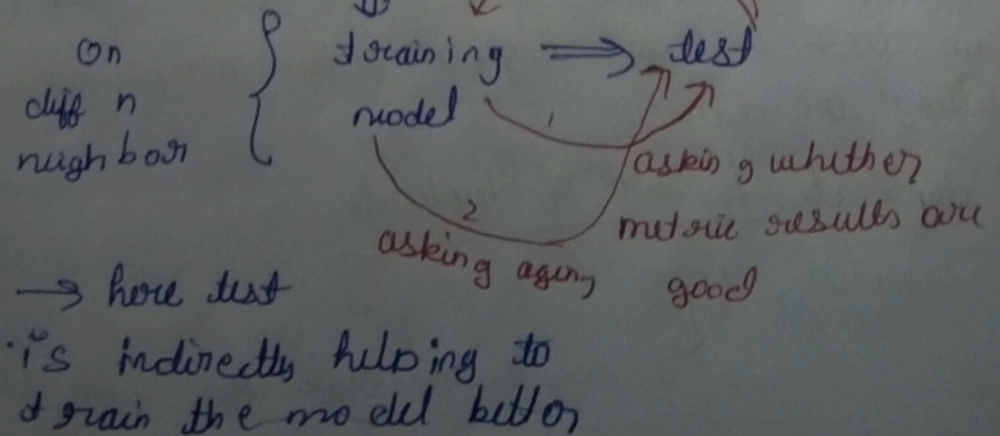
Ex:- Knn →



viii) Sampling Leakage

→ SMOTE before

splitting [under sampling or oversampling]



ix) Human / Process leakage

x) Feature Engineering leakage

How to detect:-

i) Unrealistically High Performance:-

- Very high accuracy / AUC in early experiments
- Simple model performing extremely well

Ex:- Logistic regression gives 99% accuracy on a hard problem

ii) Big gap b/w Validation and Production:-

If model's performance significantly better on the training data and validation data compared to new, unseen data, this might indicate that there's data leakage.

iii) Review Your Feature:-

- Review all the features used to train your model. Check
- Do they include any data that wouldn't be available at the prediction time
- Any data that directly / indirectly reveals target?

v) Domain Knowledge:- . does this feature make sense?
: is it logically causal?

vi) Check feature importance:-

feature importance can help to understand what model is learning. If the model places too much importance on a feature that doesn't seem directly related to the o/p, it could be a sign of leakage

feature-importance $\Rightarrow$ $f_1 \Rightarrow 0.13$
 $f_1 \Rightarrow 0.28$
 $f_3 \Rightarrow 0.81$ ←

vii) use Pipeline before:-

- preprocessing
- Cross-validation
- hyperparameter tuning

} use pipeline for real-world ML workflows.

How to remove data leakage:-

i) Understand the data and the task:-

- Before starting with any kind of data processing or modelling, It is important to understand the problem & data and how the data is collected.
- You should understand Each feature in your data & whether it will be available at the time of prediction.

ii) Careful feature Selection:-

- Review all the features used in your model
- If any feature includes information that wouldn't be available at the time of prediction or directly or indirectly gives away the target variable, it should be removed

iii) Proper data Splitting:-

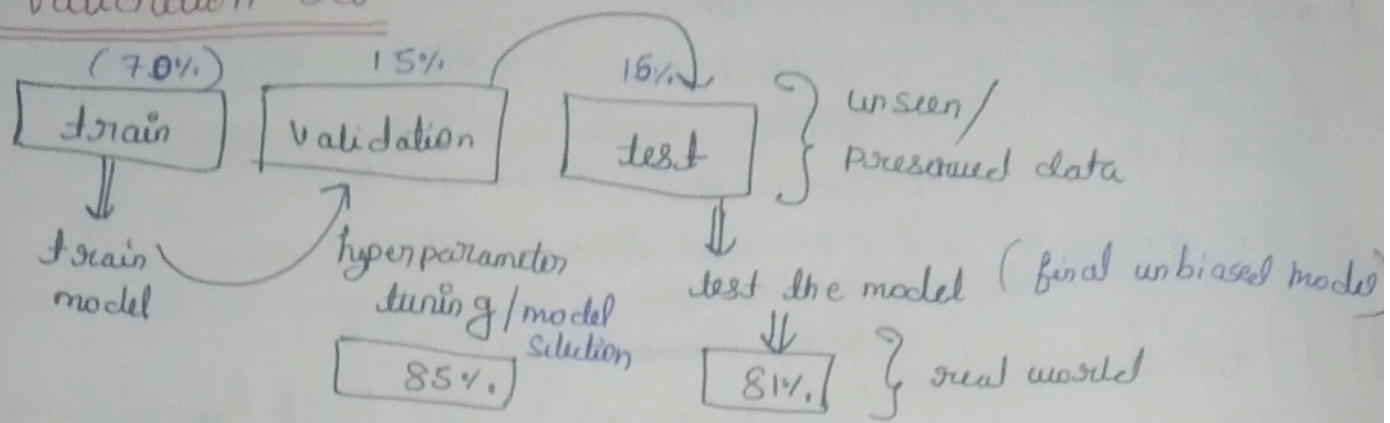
- Always split your data into training, validation at an early stage of your pipeline, before doing any feature Extraction / pre-processing.

Correct way:- `pipeline = make_pipeline(normalize, model)`
`cross_val_score(pipeline, X, y, cv=5)`

iv) Avoid Overlapping Data:- [duplicates]

- If the same features / individuals appear in both your training and test sets, this cause data leakage.
- It's important to ensure that the training and test sets represent separate, non-overlapping data instances

Validation Set

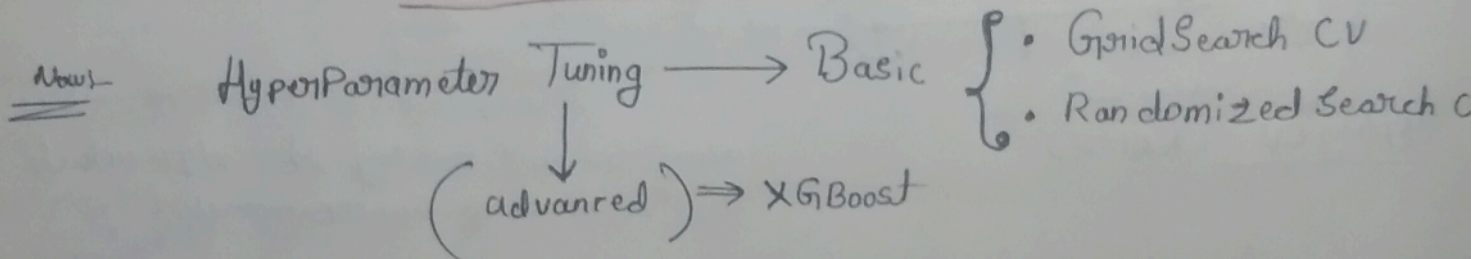


→ A validation set is used to tune hyperparameters and select models, while test set is reserved for final unbiased evaluation.

Why do we need validation set:- Validation set is used for

- Hyperparameter tuning (depth, c, k, learning rate)
- threshold selection
- model selection (logistic vs SVM vs RF)
- Early stopping

HyperParameter Tuning:-



Parameter

- Parameters are values learned automatically by the model during training.
 - They are updated using the data
- Ex:- Linear regression → coefficients
Neural Networks → weights, bias

Hyper-Parameter

- Hyperparameters are values set before training that control how the model learns.
 - They are not learned from data
- Ex:- Learning Rate,
C in logistic Regression
k in knn
max-depth in Decision Tree

why the word hyper:- Hyper $\Rightarrow$ above / beyond

Hyper parameters :-

- Control how the ~~model~~ model is trained
- decide model behaviour
- Chosen before training starts

(Settings knob)

$\rightarrow$ They operate at a higher level than parameters.

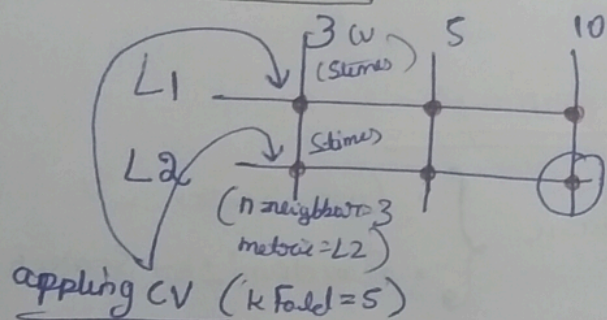
#1] Grid Search CV:- [Brute force approach]

Grid Search CV is a method to find the best hyperparameters by trying all possible combinations and Evaluating them using cross validation.

Ex: 1000 data

Cgpa	iq	Placement
10	100	1
9	90	0
8	80	1

$\Rightarrow$ Y/N model $\Rightarrow$ KNN $\Rightarrow$ N-neighbor $\Rightarrow$ 3, 5, 10...
metric $\Rightarrow$ L1, L2
 $\downarrow$
for new Cgpa, iq $\Rightarrow$ { which combo will give best results }



training 6 models
 $6 \times 5 = 30$

$\rightarrow$ Computational Expensive when using Gradient Boosting like algo which has more no. of hyperparameters (32)

$\rightarrow$ Grid Search CV performs an Exhaustive search over a predefined set of hyperparameters using cross-validation to select the best model configuration.

Limitations:

- Computationally Expensive $\Rightarrow$ Tries all Combination
- Not Scalable $\Rightarrow$ Large grid explodes
- Misses good values $\Rightarrow$ Only checks given grid

when to use

- $\rightarrow$ small datasets
- $\rightarrow$ few hyperparameters
- $\rightarrow$ baseline models

Randomized Search CV:

Randomized Search CV searches for the best hyperparameters by randomly ~~some~~ sampling combinations instead of trying all of them.

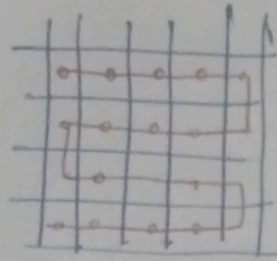
Ex:- instead 108 possibilities

↓

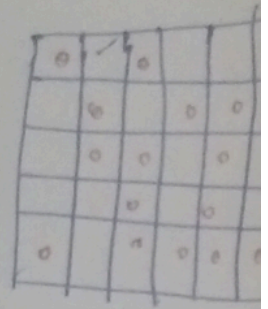
X Possibilities

↓

10/20/30/40, 50



instead of all
Combo



How it works

- define range or distributions of hyperparameters
- Randomly sample n-iter combinations.
- For Each Combination $\Rightarrow$ perform k-fold CV
- Pick best results

advantages

- Faster than Grid Search
- works well for large datasets
- Tries only few combinations
- used widely in real projects

disadvantages

- # $\Rightarrow$ may miss the best hyperparameters
- $\Rightarrow$ Results depend on randomness
- $\Rightarrow$ No guarantee of optimal solution

Hyperparameter Tuning

(analogy:- Creating best dish
picking correct ingredients & mass
(on prepare best dish))

Grid Search CV

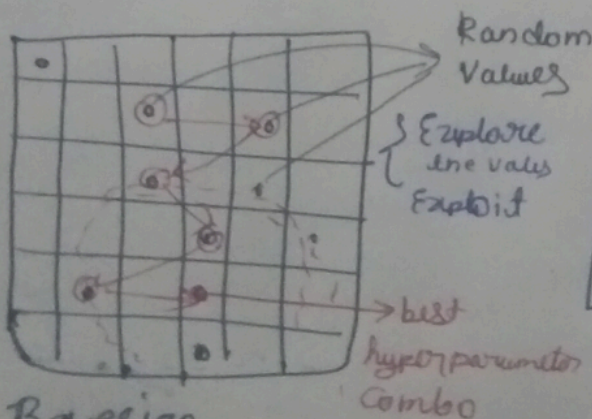
↓
{ Slow Speed
Computationally
Expensive }

Randomized CV

Search
{ high speed, but
on luck }

Advanced Technique

Bayesian Optimization



Bayesian

Bayesian optimization
learns from
previous trials
to choose
better hyperparameters

- Optuna
- Hyperopt
- Scikit-Optimize
- i) Early Stopping
- ii) Hyperband
- iv) AutoML